

Competitive Programming

Section 49 Editorial: Convex Hull Trick And Li Chao Tree

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Dynamic Line Minimum	4
2.1	Restatement	4
2.2	Algorithm and proof	4
2.3	Complexity	4
2.4	Full C++ solution	4
2.5	Full Python solution	4
3	B. Quadratic CHT DP	5
3.1	Restatement	5
3.2	Algorithm and proof	5
3.3	Complexity	5
3.4	Full C++ solution	5
3.5	Full Python solution	5
4	C. Dynamic Line Maximum	6
4.1	Restatement	6
4.2	Algorithm and proof	6
4.3	Complexity	6
4.4	Full C++ solution	6
4.5	Full Python solution	6
5	D. Line Winner	7
5.1	Restatement	7
5.2	Observations	7
5.3	Algorithm	7
5.4	Correctness proof	7
5.5	Complexity	8
5.6	Implementation notes	8
5.7	Full C++ solution	8
5.8	Full Python solution	8
5.9	Tests that matter	9
6	Testing discussion	10

1 Editorial overview

1.1 Problem-specific proof map

A. Dynamic Line Minimum. A Li Chao node keeps the line best at its midpoint; the only line that can beat it on one half is recursed into that half, so every query path sees the minimum candidate.

B. Quadratic CHT DP. Expanding the square turns each earlier state into a line queried at $x[i]$. Thus the Li Chao answer is exactly the minimizing DP transition plus the query-only terms.

C. Dynamic Line Maximum. Negating all line values reduces maximum queries to the same minimum-line invariant used in A.

1.2 Detailed solution guide

A. Static Lines, Minimum Query. Each inserted line represents one candidate linear function $y = mx + b$. A query asks for the minimum value among all candidates at a given x . The local solution evaluates all lines so the definition is visible in the code. In larger constraints, the same set of lines is stored in a convex hull or Li Chao tree so the minimum is found faster.

B. DP to Lines. A common transition is $dp[i] = \min_j(dp[j] + a[j] * x[i] + b[j])$. For each previous j , create a line with slope $a[j]$ and intercept $dp[j] + b[j]$; querying at $x[i]$ gives exactly the best transition into i . The proof is just algebra: every possible previous state becomes one line, and no line represents an illegal state.

C. Li Chao Tree Query Model. A Li Chao tree stores, for each segment of the coordinate domain, a line that is currently best at that segment midpoint. Insertion compares the new line and old line at endpoints and midpoint, keeps the better midpoint line, and recurses only where the other line can still win. The local code may scan all lines, but the invariant explains why the optimized tree answers point queries in logarithmic time.

Common pitfalls. Equal slopes should keep only the better intercept. Use 64-bit arithmetic or wider if coordinates and slopes are large. Decide up-front whether the task asks for minimum or maximum; signs flip easily.

The common model is a set of lines.

- A and C maintain lines $y=mx+b$ and query the minimum or maximum at a point. The local limits allow a direct scan, which is the clearest correctness reference.
- B expands $(x_i - x_j)^2$ so each previous DP state j becomes a line with slope $-2x_j$ and intercept $dp[j] + x_j^2$.

For larger external tasks, replace the direct scan by a Li Chao tree or a monotone convex hull. The mathematical reduction from candidate states to lines is unchanged.

2 A. Dynamic Line Minimum

2.1 Restatement

Add lines $y=mx+b$ and answer minimum value at query points.

2.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

2.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Line{long long m,b; long long get(long long x){return m*x+b;}};
```

2.5 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    q=data[0]; idx=1; lines=[]; out=[]
    for _ in range(q):
        t=data[idx]; idx+=1
        if t==1:
            m,b=data[idx],data[idx+1]; idx+=2; lines.append((m,b))
        else:
            x=data[idx]; idx+=1; out.append(str(min(m*x+b for m,b in lines)))
    print("\n".join(out))
if __name__=="__main__": main()
```

3 B. Quadratic CHT DP

3.1 Restatement

Compute $dp[i] = \min_j dp[j] + (x_i - x_j)^2 + c$ for increasing x .

3.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

3.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;long long
```

3.5 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,c=data[0],data[1]; x=data[2:]; dp=[0]*n
    for i in range(1,n):
        dp[i]=min(dp[j]+(x[i]-x[j])**2+c for j in range(i))
    print(dp[-1])
if __name__=="__main__": main()
```

4 C. Dynamic Line Maximum

4.1 Restatement

Add lines and answer maximum value at query points.

4.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

4.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Line{long long m,b; long long get(long long x){return m*x+b;}};
```

4.5 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    q=data[0]; idx=1; lines=[]; out=[]
    for _ in range(q):
        t=data[idx]; idx+=1
        if t==1:
            m,b=data[idx],data[idx+1]; idx+=2; lines.append((m,b))
        else:
            x=data[idx]; idx+=1; out.append(str(max(m*x+b for m,b in lines)))
    print("\n".join(out))
if __name__=="__main__": main()
```

5 D. Line Winner

5.1 Restatement

Lines are inserted online. At a query coordinate, print the smallest line value and, among equal values, the earliest inserted line. This blind exercise adds a deterministic tie rule to dynamic line minimum queries.

5.2 Observations

Compare a line at x by the pair $(m \cdot x + b, \text{insertion_id})$. Lexicographic pair comparison exactly implements both the value objective and the tie rule. Read all operations first and compress the query coordinates; a Li Chao tree then needs nodes only for coordinates that will be queried.

At a segment, retain the better line at its midpoint. The other line can only win on one side, so recurse into that side after comparing endpoints. This is the Li Chao invariant, unchanged except that every comparison uses the pair.

5.3 Algorithm

```
collect and sort distinct query x values
for every inserted line, assign its insertion id
insert it into a Li Chao segment tree using pair comparisons at endpoints/mid
for a query x, take the minimum pair over tree nodes on x's root-to-leaf path
```

5.4 Correctness proof

Lemma. Every Li Chao node retains a line that is best at its midpoint among lines routed through that node, while any line that can improve another point is passed to the only child interval where it can do so.

Proof. The midpoint comparison retains the better pair. The difference of two linear values is linear; with lexicographic id resolving equality, the other line can beat the retained line on at most one side. The endpoint tests route it precisely there. $\square$

Theorem. Each query returns the required value and earliest attaining id.

Proof. By the lemma, any inserted line that can win at the query coordinate is retained on some node of that coordinate's search path. Taking the minimum pair over that path considers every possible winner and chooses exactly the specified lexicographic optimum. $\square$

5.5 Complexity

For K distinct query coordinates, insertion and query cost $O(\log K)$; memory is $O(K)$. Products require long long under the local bounds.

5.6 Implementation notes

In C++, represent an empty line as $(\text{INF}, \text{INT_MAX})$. In Python, use a matching large pair. Do not use only a value comparison on ties: a later equal line must not replace an earlier one.

5.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using ll=long long; const ll INF=(1LL<<62);
struct Line{ll m=0,b=INF;int id=INT_MAX; pair<ll,int> at(ll x)const{return {m*x+b,id};}};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int q;if(!(cin>>q))return 0;vector<a
```

5.8 Full Python solution

```
import sys
INF=10**30
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));
    if not d:return
    q=d[0];i=1;ops=[];xs=[]
    for _ in range(q):
        t=d[i];a=d[i+1];i+=2
        if t==1:b=d[i];i+=1;ops.append((t,a,b))
        else:ops.append((t,a,0));xs.append(a)
    xs=sorted(set(xs));where={x:i for i,x in enumerate(xs)};tree=[None]*(4*len(xs))
    def val(line,x):return (line[0]*x+line[1],line[2]) if line else (INF,10**9)
    def add(line,p,l,r):
        if tree[p] is None:tree[p]=line;return
        m=(l+r)//2
        if val(line,xs[m])<val(tree[p],xs[m]):tree[p],line=line,tree[p]
        if l==r:return
        if val(line,xs[l])<val(tree[p],xs[l]):add(line,p*2,l,m)
        elif val(line,xs[r])<val(tree[p],xs[r]):add(line,p*2+1,m+1,r)
    def get(p,l,r,k):
        ans=val(tree[p],xs[k]);
        if l==r:return ans
        m=(l+r)//2;return min(ans,get(p*2,l,m,k) if k<=m else get(p*2+1,m+1,r,k))
    out=[];id=0
    for t,a,b in ops:
        if t==1:id+=1;add((a,b,id),1,0,len(xs)-1)
```

```
    else:out.append('%d %d'%get(1,0,len(xs)-1,where[a]))
print('\n'.join(out))
if __name__=='__main__':main()
```

5.9 Tests that matter

Several identical lines: always select the first id.
Lines intersecting at a query coordinate.
Negative x and negative slopes.
100,000 equal insertions followed by 100,000 queries.

6 Testing discussion

Stress every line query against evaluating all inserted lines. Equal slopes need an explicit keep-better-intercept rule. For quadratic DP, expand $(x_i - x_j)^2$ before choosing slope and intercept; a sign error changes min to max behavior. Test repeated x , negative values, and a single line.

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.